WBS TRAINING

Handout

TF-US-PSS

Einführung in die strukturierte Softwareentwicklung

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

Inhalt

1 Einführung PSS 3
1.1 Bedeutung strukturierter Planung 3
1.1.1 Einführung und Kontext 3
1.1.2 Praxisbeispiel 3
1.1.3 Erklärender Text 5
1.1.4 Fragen zur Vertiefung 8
1.2 Rolle von Modellierung und Pseudocode 9
1.2.1 Einführung und Kontext 9
1.2.2 Praxisbeispiel 10
1.2.3 Erklärender Text 11
1.2.4 Fragen zur Vertiefung 13
1.3 Überblick über den Softwareentwicklungsprozess 14
1.3.1 Einführung und Kontext 14
1.3.2 Praxisbeispiel 14
1.3.3 Erklärender Text 15
1.3.4 Vorgehensmodelle im Überblick 18
1.3.5 Fragen zur Vertiefung 18

Dieses Handout wurde mit Unterstützung von KI entwickelt.

1 Einführung PSS

1.1 Bedeutung strukturierter Planung

1.1.1 Einführung und Kontext

Die **strukturierte Softwareentwicklung** bildet das **unverzichtbare Fundament** für die gesamte professionelle Informationstechnologie (IT). In der heutigen digitalen Wirtschaft, in der Unternehmensprozesse, kritische Infrastrukturen und die Kundenschnittstellen maßgeblich von Software abhängen, sind die Anforderungen an diese Programme weitaus höher als an private oder Hobby-Projekte.

Während ein privates Projekt möglicherweise nach dem Prinzip "Try-and-Error" erstellt werden kann, muss **Unternehmenssoftware** folgende elementare Kriterien erfüllen:

- **Zuverlässigkeit:** Die Anwendung muss ihre zugesicherten Funktionen fehlerfrei und konsistent ausführen, insbesondere in kritischen Geschäftsprozessen.
- **Wartbarkeit:** Der Code muss für andere Entwickler (oder das zukünftige Ich des ursprünglichen Entwicklers) verständlich und modifizierbar sein, um Fehler zu beheben oder Anpassungen vorzunehmen.
- **Skalierbarkeit:** Die Software muss in der Lage sein, mit wachsenden Benutzerzahlen, Datenmengen und komplexeren Anforderungen umzugehen, ohne dass die Performance signifikant einbricht.

Diese zentralen Qualitätsmerkmale sind **ohne eine tiefgehende, durchdachte und disziplinierte Planung** – oft auch als **Software-Engineering-Prozess** bezeichnet – **schlicht nicht realisierbar**. Die Planungsphase transformiert eine bloße Idee in ein robustes, langlebiges und wertschöpfendes IT-Asset, indem sie Unsicherheiten reduziert, Risiken managt und einen klaren Fahrplan für das Entwicklungsteam vorgibt. Sie ist somit der **Grundstein für langfristigen Projekterfolg** und die **Minimierung der Gesamtkosten** einer Anwendung über ihren gesamten Lebenszyklus.

1.1.2 Praxisbeispiel

Zur Veranschaulichung der Notwendigkeit strukturierter Planung dient das folgende, häufig anzutreffende Szenario in der Ausbildung oder in kleineren Entwicklungsteams:

Ein Auszubildender im Bereich Fachinformatik soll eine Anwendung zur Verwaltung von Kundendaten entwickeln (**CRUD-Anwendung**). Ohne Planung beginnt er direkt mit dem Programmieren. Er nutzt globale Variablen, mischt die Geschäftslogik mit der Benutzeroberfläche und kommentiert kaum.

Nach kurzer Zeit wird der Code **unübersichtlich** (**Spaghetti-Code**), Fehler häufen sich (z. B. fehlerhafte Datenkonsistenz) und Änderungen an der Datenbank sind kaum möglich, da sie den gesamten Code betreffen würden. Er ist im sogenannten **"Technical Debt"** (technische Schulden) gefangen.

Erläuterung des Szenarios und der Fachbegriffe

- **CRUD-Anwendung:** Dies ist eine fundamentale Kategorie von Software. **CRUD** ist ein Akronym für die vier Grundfunktionen, die bei der Verwaltung persistenter Daten (in der Regel in einer Datenbank) notwendig sind:
    - **C**reate (Erstellen): Neue Datensätze hinzufügen (z. B. einen neuen Kunden anlegen).
    - **R**ead (Lesen): Bestehende Daten abrufen und anzeigen (z. B. eine Kundenliste anzeigen).
    - **U**pdate (Aktualisieren): Bestehende Datensätze ändern (z. B. die Adresse eines Kunden korrigieren).
    - **D**elete (Löschen): Datensätze entfernen (z. B. einen Kunden aus der Datenbank löschen).
- **Ohne Planung beginnt er direkt mit dem Programmieren:** Der Auszubildende verfällt in den sogenannten **"Code-and-Fix"**-Ansatz, eine Vorgehensweise, die in der professionellen Entwicklung als **Anti-Pattern** gilt. Die Versuchung ist groß, sofort Ergebnisse zu sehen, doch die langfristigen Konsequenzen sind meist fatal.
- **Er nutzt globale Variablen:** Eine **globale Variable** ist eine Variable im Code, die *überall* in der Anwendung aufgerufen und verändert werden kann. Dies steht im Gegensatz zu **lokalen Variablen**, die nur innerhalb eines begrenzten Bereichs (z. B. einer Funktion) gültig sind. Die Verwendung globaler Variablen erschwert die Nachverfolgung, wo und wann Daten geändert werden, was die Fehlersuche drastisch kompliziert.
- **Mischt die Geschäftslogik mit der Benutzeroberfläche:** Die **Geschäftslogik** (oder Domänenlogik) beinhaltet die Regeln und Algorithmen, die den eigentlichen Geschäftswert der Anwendung definieren (z. B. wie ein Rabatt berechnet wird). Die **Benutzeroberfläche** (User Interface, UI) ist der Teil, mit dem der Benutzer interagiert (z. B. Eingabefelder und Schaltflächen). Die Vermischung beider Bereiche widerspricht dem **Prinzip der Trennung der Belange (Separation of Concerns)** und führt dazu, dass jede Änderung an einem Teil den anderen unnötigerweise beeinflusst.
- **Code unübersichtlich (Spaghetti-Code): Spaghetti-Code** ist ein informeller, aber weit verbreiteter Begriff für Quellcode, dessen Struktur unorganisiert, kaum

kommentiert und voller Sprünge (im übertragenen Sinne) zwischen verschiedenen Codebereichen ist. Er ist chaotisch, schwer zu folgen und noch schwerer zu warten – analog zu einem Teller Spaghetti, bei dem es unmöglich ist, den Anfang oder das Ende eines einzelnen Fadens zu finden.

- **Fehler häufen sich (z. B. fehlerhafte Datenkonsistenz): Datenkonsistenz** beschreibt den Zustand, dass Daten innerhalb eines Systems korrekt, widerspruchsfrei und gültig sind. Eine **fehlerhafte Datenkonsistenz** liegt vor, wenn beispielsweise in der Kundendatenbank ein Kunde einmal mit PLZ 10115 und einmal mit 90403 gespeichert ist oder wenn ein Kunde zwar gelöscht wurde, aber seine Bestellhistorie noch vorhanden ist und auf eine nicht mehr existierende ID verweist.
- **Er ist im sogenannten "Technical Debt" (technische Schulden) gefangen: Technical Debt** ist eine Metapher aus der Softwareentwicklung. Sie beschreibt die **zusätzlichen Kosten** für zukünftige Entwicklungsarbeiten, die durch die Wahl einer einfachen, aber nicht optimalen Lösung *jetzt* entstehen. Es ist wie die Aufnahme eines Kredits (der schnelle Code ohne Planung) – man bekommt kurzfristig einen Vorteil (die App ist schnell fertig), muss aber langfristig hohe Zinsen (zusätzliche Zeit für Fehlerbehebung, Wartung und Umbau) zahlen.

Die Lösung durch Struktur

Erst durch eine **strukturierte Planung** mit klaren **Anforderungen** (was soll das System können?), **Modularisierung** (**Trennung von Zuständigkeiten** in unabhängige, testbare Bausteine), einem dedizierten **Entwurf** (Design der Architektur und Datenstruktur) und einer **Dokumentation** (schriftliche Festhaltung von Entscheidungen und Funktionen) gelingt es, eine **funktionierende** und vor allem **wartbare** Lösung zu erstellen. Die Planungsphase hätte hier von vornherein die Probleme der Globalen Variablen und der Vermischung von Logik und UI verhindert.

1.1.3 Erklärender Text

Die strukturierte Planung ist nicht nur eine formelle Anforderung, sondern die **unabdingbare Grundlage jeder professionellen Softwareentwicklung**. Sie dient primär der **Qualitätssicherung** – dem systematischen Prozess zur Sicherstellung, dass die Software die Anforderungen erfüllt – und dem **Risikomanagement**, der aktiven Identifizierung, Analyse und Kontrolle potenzieller Projektrisiken.

Die folgenden Dimensionen der Planung belegen ihren entscheidenden Mehrwert:

Dimension	Vorteil der Strukturierung	Erklärung
Klarheit der Anforderungen	Anforderungsmanagement	Verhindert "Scope Creep". Dokumentiert genau, was die Software leisten soll.
Logischer Programmablauf	Wartbarkeit und Lesbarkeit	Nutzung von Prinzipien wie Modularität und **KISS** zur Schaffung transparenten Codes.
Frühe Fehlererkennung	Kostensenkung	Fehler in der Planungsphase sind deutlich günstiger zu beheben als nach der Veröffentlichung.
Anpassungsfähigkeit	Skalierbarkeit	Klare **Architekturmuster** erlauben Austausch und Erweiterung von Komponenten ohne Systemneuschreibung.
Teamarbeit	Kollaboration und Standardisierung	**Coding Conventions** und eine gemeinsame Struktur ermöglichen paralleles, effektives Arbeiten.

Detaillierte Erläuterung der Vorteile

Klarheit der Anforderungen: Anforderungsmanagement

Anforderungsmanagement ist der systematische Prozess, bei dem die Bedürfnisse der Stakeholder (Interessengruppen) ermittelt, dokumentiert, analysiert, verfolgt und priorisiert werden.

- **Vorteil:** Dies ist der effektivste Schutz vor dem Phänomen des **"Scope Creep"**.
- **Erläuterung: Scope Creep** (wörtlich: schleichender Projektumfang) beschreibt die **unaufhaltsame, unkontrollierte Ausweitung des Projektumfangs** nach Projektbeginn. Wenn Anforderungen nicht zu Beginn exakt definiert und schriftlich festgehalten werden, kommen im Laufe der Entwicklung ständig neue, nicht budgetierte Funktionen hinzu. Dies führt unweigerlich zu Projektverzögerungen und Kostenüberschreitungen.
- **Instrument:** Das **Lastenheft** ist ein zentrales Dokument, das **aus Sicht des Auftraggebers** (Was will ich?) präzise dokumentiert, welche Funktionen die Software **leisten soll**. Es bildet die Basis für den Vertrag zwischen Auftraggeber und Auftragnehmer (Entwicklungsteam).

Logischer Programmablauf: Wartbarkeit und Lesbarkeit

Ein logisch und strukturiert aufgebauter Programmablauf ist direkt verantwortlich für die **Wartbarkeit** und die **Lesbarkeit** des Codes.

- **Vorteil:** Der Code wird transparent und leicht verständlich.
- **Erläuterung:** Dies wird durch die Anwendung zentraler Entwurfsprinzipien erreicht:
    - **Modularität (Trennung in unabhängige Bausteine):** Der Gesamtcode wird in kleine, in sich geschlossene Einheiten (Module, Klassen, Funktionen) zerlegt, die jeweils nur eine klar definierte Aufgabe erfüllen. Dies erleichtert das Testen, die Wiederverwendung und die lokale Fehlerbehebung.
    - **KISS (Keep it Simple, Stupid):** Ein fundamentales Entwurfsprinzip, das besagt, dass die meisten Systeme am besten funktionieren, wenn sie einfach gehalten werden und nicht unnötig komplex. Entwickler sollen immer die einfachste, klarste Lösung wählen.

Frühe Fehlererkennung: Kostensenkung

Die strukturierte Planung integriert Mechanismen zur Fehlererkennung, noch bevor der erste Code geschrieben wird.

- **Vorteil:** Massiv reduzierte Kosten für Fehlerbehebung.
- **Erläuterung:** Zahlreiche Studien und Praxiserfahrungen belegen, dass **Fehler, die bereits in der Planungs- oder Entwurfsphase (z. B. durch Reviews von Architekturdokumenten) erkannt und korrigiert werden, um ein Vielfaches (teilweise bis zu 100-mal) günstiger zu beheben sind** als Fehler, die erst nach dem **Deployment** (der Veröffentlichung bzw. der Produktivsetzung der Software beim Kunden oder auf dem Markt) entdeckt werden. Ein Fehler in der Konzeption erfordert nur eine Änderung eines Dokuments, wohingegen ein Fehler in der Produktionsumgebung eine aufwändige Fehleranalyse, Code-Änderung, Neu-Testung, Neu-Verteilung und möglicherweise einen direkten Schaden beim Kunden nach sich zieht.

Anpassungsfähigkeit: Skalierbarkeit

Ein gut geplanter Software-Entwurf stellt die Weichen für die Zukunftsfähigkeit des Systems.

- **Vorteil:** Das System kann mit steigenden Anforderungen und Lasten mühelos wachsen.

- **Erläuterung:** Hierfür werden klare **Architekturmuster** angewendet. Das bekannteste Beispiel ist die **3-Schicht-Architektur**. Diese trennt die Anwendung in drei logische, voneinander unabhängige Bereiche:
    1. **Präsentationsschicht (UI):** Die Benutzeroberfläche.
    2. **Logikschicht (Business Logic):** Die eigentlichen Geschäftsregeln.
    3. **Datenhaltungsschicht (Persistence):** Die Datenbank. Durch diese klare Trennung können einzelne Komponenten (z. B. die Datenbank von einer relationalen auf eine NoSQL-Datenbank) **ausgetauscht oder erweitert** werden, ohne dass die gesamte Anwendung neu geschrieben werden muss. Dies ist das Kernprinzip der Skalierbarkeit und Langlebigkeit.

Teamarbeit: Kollaboration und Standardisierung

In den seltensten Fällen arbeitet ein Entwickler allein an komplexer Unternehmenssoftware. Planung schafft die notwendigen Rahmenbedingungen für effektive Zusammenarbeit.

- **Vorteil:** Effiziente, parallele Entwicklung durch mehrere Personen.
- **Erläuterung:** Eine gemeinsame, **vereinheitlichte Struktur** (z. B. durch einheitliche Ordnerstrukturen und benannte Module) und vor allem **Coding Conventions** (Programmierrichtlinien) sind essenziell. **Coding Conventions** sind ein Satz von Regeln, die festlegen, wie Code zu formatieren ist (z. B. Einrückungen, Benennung von Variablen, Platzierung von Klammern). Sie stellen sicher, dass Code von verschiedenen Entwicklern so aussieht, als wäre er von einer einzigen Person geschrieben worden. Dies ist **unerlässlich**, damit mehrere Entwickler **parallel** an unterschiedlichen Teilen eines Projekts arbeiten können, ohne sich gegenseitig zu blockieren oder durch inkonsistenten Stil die Lesbarkeit zu

1.1.4 Fragen zur Vertiefung

1. Warum ist Planung vor dem Programmieren wichtig (Stichwort: Kosten)?

   Planung ist entscheidend für die **Kostensenkung**. Fehler, die in der Planungs- oder Entwurfsphase erkannt und behoben werden, sind laut Canvas bis zu **100-mal günstiger** als Fehler, die erst nach dem **Deployment** (Veröffentlichung) entdeckt werden. Planung dient somit dem **Risikomanagement**.

2. Welche Probleme können ohne strukturierte Planung entstehen? Nennen Sie zwei Begriffe aus dem erläuternden Text (Spaghetti-Code, Technical Debt).

Ohne Planung entsteht schnell **unübersichtlicher Code** (**Spaghetti-Code**), der schwer zu verstehen und zu warten ist. Dies führt zur Anhäufung von **Technical Debt** (technischen Schulden), wodurch zukünftige Änderungen und Erweiterungen teuer und kompliziert werden.

3. Welche Vorteile hat strukturierte Softwareentwicklung für Teams und die langfristige Lebensdauer des Produkts?

   Für **Teams** sorgt sie für **Kollaboration** und **Standardisierung** durch klare Prozesse und **Coding Conventions**. Für die **langfristige Lebensdauer** des Produkts gewährleistet sie **Wartbarkeit**, **Lesbarkeit** und **Skalierbarkeit** durch Prinzipien wie **Modularität** und klare Architekturmuster (z. B. 3-Schicht-Architektur).

1.2 Rolle von Modellierung und Pseudocode

1.2.1 Einführung und Kontext

Der Bauplan Der Softwarearchitektur

Die Entwicklung komplexer **Software** (Programme und Datenstrukturen, die es Computern ermöglichen, bestimmte Aufgaben auszuführen) ist ein anspruchsvoller **Ingenieursprozess** (die Anwendung wissenschaftlicher und mathematischer Prinzipien zur Entwicklung und Wartung von Systemen), der weitreichenden Parallelen zum Bauwesen aufweist.

Bevor ein Architekt ein Haus baut, erstellt er detaillierte Baupläne. In der professionellen Softwareentwicklung übernehmen **Modellierungssprachen** (standardisierte, visuelle Sprachen zur Darstellung von Systemstrukturen und -abläufen) und **Pseudocode** (eine informelle Beschreibung von Algorithmen, die Programmiersprachenstrukturen nachahmt) exakt diese Planungsrolle. Sie stellen die notwendigen **Abstraktionswerkzeuge** dar.

Der entscheidende Zweck dieser Werkzeuge ist die **Trennung der Logik von der Implementierung**:

1. **Logik (Das Was):** Die abstrakte Definition dessen, *was* die Software tun soll, wie sie strukturiert ist und welche **Prozesse** (eine Reihe von aufeinanderfolgenden Aktionen oder Schritten zur Erreichung eines Ziels) sie ausführt.

2. **Implementierung (Das Wie):** Die konkrete **Umsetzung** der Logik in einer spezifischen **Programmiersprache** (ein formalisiertes Vokabular und eine Reihe von Anweisungen, mit denen ein Computerprogramme geschrieben werden) unter Berücksichtigung der **Syntax** (der grammatikalischen Regeln der Sprache) und technischen Details.

Diese Trennung ist fundamental für die **Qualitätssicherung** (der systematische Prozess zur Überwachung, Überprüfung und Sicherstellung, dass ein Produkt die festgelegten Anforderungen erfüllt) und die Vermeidung von Fehlern in späten, teuren Projektphasen.

1.2.2 Praxisbeispiel

Ein Entwicklerteam plant eine neue Webanwendung. Bevor der erste Code geschrieben wird, erstellen sie ein **Anwendungsfalldiagramm** (**Use Case Diagramm**) als erste Übersicht über die Nutzerinteraktion. Für die interne Struktur erstellen sie ein **Klassendiagramm** (Datenmodelle) und zur Abbildung komplexer Abläufe ein **Aktivitätsdiagramm** (Prozess-Flow). Danach beschreiben sie die wichtigsten Algorithmen in **Pseudocode**, um die Logik zu validieren. Erst dann beginnt die eigentliche Programmierung in einer spezifischen Sprache (z. B. Java oder Python).

Der Systematische Entwurf Einer Webanwendung

Ein **Entwicklerteam** (eine Gruppe von Fachleuten, die gemeinsam an der Erstellung und Wartung von Software arbeitet) plant eine neue **Webanwendung** (eine Software, die über einen Webbrowser zugänglich ist und typischerweise auf einem Server ausgeführt wird).

Der strukturierte Prozess, der vor dem eigentlichen Codieren beginnt, umfasst folgende Schritte, die alle auf visueller Modellierung basieren:

1. **Anwendungsfalldiagramm (Use Case Diagramm):** Dieses Diagramm, ein Teil der UML, dient als **erste Übersicht über die Nutzerinteraktion**. Es visualisiert die **Akteure** (externe Entitäten, wie Benutzer oder andere Systeme, die mit dem System interagieren) und die **Anwendungsfälle** (**Use Cases**), welche die Funktionen oder Dienstleistungen darstellen, die das System den Akteuren bietet (z. B. "Daten eingeben", "Bericht erstellen"). Es ist der erste Schritt der **Anforderungsanalyse**.
2. **Klassendiagramm (Datenmodelle):** Dies ist das wichtigste Diagramm der **statischen Modellierung**. Es beschreibt das **Datenmodell** der Anwendung, also die interne Struktur der Daten und des Codes. Es definiert:
    - **Klassen:** Die Baupläne für Objekte, die zusammengehörige Daten (**Attribute**) und Funktionalitäten (**Methoden**) bündeln.
    - **Attribute:** Die Eigenschaften oder Datenfelder einer Klasse (z. B. der Name oder das Alter eines "Kunden").
    - **Beziehungen:** Wie Klassen miteinander in Verbindung stehen (z. B. *Vererbung*, bei der eine Klasse Eigenschaften einer anderen übernimmt; oder *Assoziationen*, die logische Verbindungen darstellen). Es bildet die logische Grundlage für die spätere **Datenbankstruktur**.
3. **Aktivitätsdiagramm (Prozess-Flow):** Dieses Diagramm wird zur Abbildung **komplexer Abläufe** oder **Geschäftslogiken** (die Regeln und Algorithmen, die den Wert

des Unternehmens widerspiegeln) verwendet. Es visualisiert den **Prozess-Flow** (den schrittweisen, logischen Ablauf der Aktivitäten) ähnlich einem Flussdiagramm. Es ist ideal, um **Entscheidungspunkte** und parallele Ausführungspfade zu dokumentieren.

4. **Pseudocode zur Logikvalidierung:** Bevor die **spezifische Programmiersprache** (z. B. Java oder Python) gewählt und verwendet wird, werden die wichtigsten **Algorithmen** (eine eindeutige, endliche Folge von Anweisungen oder Schritten zur Lösung eines Problems) in **Pseudocode** beschrieben. Dies dient der **Validierung der Logik**, um sicherzustellen, dass der gedachte Lösungsweg des Algorithmus konzeptionell korrekt ist.

Erst dann beginnt die eigentliche Programmierung in einer **spezifischen Sprache** (z. B. **Java** oder **Python**, beides verbreitete Hochsprachen für verschiedene Anwendungsbereiche).

1.2.3 Erklärender Text

Essenzielle Abstraktionswerkzeuge

Modellierung und **Pseudocode** sind die **essenziellen Abstraktionswerkzeuge** in der **Softwaretechnik** (dem disziplinierten Ansatz zur Entwicklung von Software).

A. Modellierung

Die **Modellierung** ist der Prozess der Erstellung abstrakter Modelle eines Systems. Dies geschieht in der IT meist mithilfe der **UML (Unified Modeling Language)**.

UML (Unified Modeling Language)

Die **UML** ist die **standardisierte, grafische Modellierungssprache** im Bereich des **Software-Engineering**. Sie bietet eine Reihe von Diagrammen zur visuellen Darstellung verschiedener Aspekte eines Softwaresystems.

- **Zweck: Gemeinsames Verständnis:** Die UML hilft, **komplexe Systeme visuell zu erfassen**. Ihr Hauptzweck ist die **Schaffung eines gemeinsamen Verständnisses** (*Common Understanding*) zwischen allen **Stakeholdern** (interessierten Parteien), also allen Personen, die ein Interesse am Projekterfolg haben, wie Entwicklern, Management und Kunden.

Wichtige Diagrammtypen

UML-Diagramme werden in zwei Hauptkategorien unterteilt:

1. **Strukturdiagramme (Statische Sicht):**
    - **Beschreibung:** Sie zeigen die **statischen** Elemente (die unveränderlichen Komponenten) des Systems und deren **Beziehungen**. Sie beschreiben die physische oder logische Zusammensetzung.

- **Beispiel: Klassendiagramm** (für Datenstruktur) oder **Komponentendiagramm** (für die physische Aufteilung des Systems).

Verhaltensdiagramme (Dynamische Sicht):

- **Beschreibung:** Sie beschreiben die **dynamischen** Aspekte, d.h., wie sich das System oder seine Teile im Laufe der Zeit verhalten und **welche Abläufe** stattfinden (zur Laufzeit).
- **Beispiele:**
  - **Aktivitätsdiagramm:** Zur Darstellung von **Prozess-Flows**.
  - **Sequenzdiagramm:** Zur Abbildung der **zeitlichen Abfolge** und des Austauschs von **Nachrichten** (Kommunikation) zwischen Objekten/Komponenten.

B. Pseudocode

Pseudocode ist eine **informelle, sprachunabhängige Beschreibung** von Algorithmen. Er verwendet die grundlegende Struktur von Programmiersprachen (wie Schleifen und Bedingungen), ist aber zur besseren Lesbarkeit in natürlicher Sprache gehalten.

- **Funktion (Fokus auf Logik):** Erlaubt es Entwicklern, sich **ausschließlich auf den Algorithmus** und die logische Abfolge zu konzentrieren, **ohne sich um die Syntax** einer bestimmten Programmiersprache kümmern zu müssen. Er agiert als **Brücke** zwischen der menschlich konzipierten Lösung und dem **Maschinencode** (den binären Anweisungen, die der Computer direkt ausführt).
- **Bestandteile (Grundlegende Programmierkonzepte):**
  - **IF-THEN-ELSE (Bedingungen/Verzweigungen):** Dient zur Steuerung des Programmflusses basierend auf dem Wahrheitswert einer Aussage.
  - **WHILE/FOR (Schleifen/Iterationen):** Dient zur Beschreibung der wiederholten Ausführung eines Codeblocks, solange eine Bedingung erfüllt ist oder eine bestimmte Anzahl von Wiederholungen erreicht ist.
  - **FUNKTION/METHODE (Abläufe/Kapselung):** Benannte Codeblöcke, die eine bestimmte Aufgabe ausführen und die Wiederverwendbarkeit von Logik gewährleisten.

- **Beispiel Für Pseudocode:**

```
FUNKTION berechneSteuer(Eingabe: bruttoGehalt)
    // Beginnt eine bedingte Anweisung
    WENN bruttoGehalt > 50000 DANN
        // Weist der Variable 'steuer' einen Wert zu
        steuer = bruttoGehalt * 0.35
    SONST
        steuer = bruttoGehalt * 0.20
    ENDE WENN

    // Gibt das Ergebnis der Funktion zurück
    GIB ZURÜCK steuer
ENDE FUNKTION
```

Dieses Beispiel demonstriert die logische Struktur des Algorithmus in einer für Menschen leicht verständlichen Form.

1.2.4 Fragen zur Vertiefung

1. Was ist der Zweck von Modellierung in der Softwareentwicklung, und nennen Sie zwei wichtige UML-Diagrammtypen?

   Der Zweck der **Modellierung** (häufig mit **UML** – Unified Modeling Language) ist die **Schaffung eines gemeinsamen Verständnisses** zwischen Entwicklern, Kunden und Stakeholdern und die visuelle Darstellung komplexer Systeme. Zwei wichtige Typen sind **Klassendiagramme** (Struktur/Daten) und **Aktivitäts- oder Sequenzdiagramme** (Abläufe/Verhalten).

2. Wie hilft Pseudocode bei der Programmplanung? Worin liegt der Vorteil gegenüber dem direkten Programmieren?

   Pseudocode beschreibt die Programmlogik in einfacher, **sprachunabhängiger** Form. Der Vorteil ist, dass sich Entwickler auf den **Algorithmus** und die **Logik** konzentrieren können, ohne sich um die spezifische Syntax einer Programmiersprache kümmern zu müssen. Dies hilft, logische Fehler frühzeitig zu erkennen.

3. Erklären Sie, warum Modellierung und Pseudocode die **Kommunikation** im Team verbessern.

   Sie dienen als essenzielle **Abstraktionswerkzeuge** und eine gemeinsame Sprache. Durch die visuelle (Modellierung) und informelle (Pseudocode) Darstellung komplexer Konzepte können alle Teammitglieder das geplante System verstehen und darüber diskutieren, bevor kostspieliger Code geschrieben wird.

1.3 Überblick über den Softwareentwicklungsprozess

1.3.1 Einführung und Kontext

Der Unverzichtbare Fahrplan

Die Entwicklung von Software, unabhängig von deren Komplexität, ist kein chaotischer Prozess, sondern ein strukturierter Ablauf von logisch aufeinanderfolgenden Aktivitäten. Dieser Rahmen wird als **Software Development Life Cycle** (SDLC) oder **Software-Lebenszyklus** bezeichnet.

Der **SDLC** dient als **organisatorischer Bauplan** und stellt sicher, dass die Entwicklung eines Softwaresystems systematisch erfolgt. Er legt die Reihenfolge fest, in der die verschiedenen Aufgaben durchgeführt werden, um ein qualitativ hochwertiges und funktionsfähiges Produkt zu gewährleisten.

Dabei ist es unerheblich, welches **Vorgehensmodell** (die spezifische Methodik zur Organisation und Steuerung des gesamten Entwicklungsprozesses) gewählt wird:

- **Klassische Modelle** (z. B. das **Wasserfallmodell**): Diese sind sequenziell und linear.
- **Agile Modelle** (z. B. **Scrum** oder **Kanban**): Diese sind iterativ und inkrementell.

Obwohl die **Ausführung und die Reihenfolge** der Phasen je nach Modell variieren, **durchläuft jede professionelle Softwareentwicklung im Grunde immer dieselbe Reihe logisch aufeinanderfolgender Phasen**. Dieser **Lebenszyklus** stellt somit sicher, dass **kein entscheidender Schritt** – von der Konzeption bis zur Auslieferung und Pflege – **vergessen wird**, und minimiert dadurch das Risiko von Fehlern und Kostenüberschreitungen.

1.3.2 Praxisbeispiel

Entwicklung Einer Mobilen App.

Zur Veranschaulichung des SDLC-Prozesses betrachten wir die Entwicklung einer neuen **App** (einer mobilen Anwendung) durch ein Technologieunternehmen:

- **Anforderungsanalyse:** Das **Produktmanagement** (die Abteilung, die für die strategische Ausrichtung und das erfolgreiche Management eines Produkts über seinen gesamten Lebenszyklus verantwortlich ist) organisiert **Workshops**. Ziel ist es, die **User Stories** festzulegen. Eine **User Story** ist eine informelle, einfache Beschreibung einer Funktion aus der Perspektive des Endbenutzers (z. B. "Als Kunde möchte ich per Fingerabdruck bezahlen können, um den Checkout-Prozess zu beschleunigen").
- **Entwurf:** Die **Architekten** (hochrangige Entwickler, die für das übergeordnete technische Design und die Struktur des gesamten Systems verantwortlich sind) legen die **technische Basis** fest. Dazu gehören Entscheidungen über die zu verwendende **Datenbank** (das organisierte Speichersystem für die Daten), die **Programmiersprache** (z. B. Swift für iOS oder Kotlin für Android) und die **API-Schnittstellen** (**Application**

Programming Interfaces, standardisierte Protokolle und Routinen, die es verschiedenen Softwarekomponenten ermöglichen, miteinander zu kommunizieren).

- **Implementierung:** Die **Entwickler** (**Programmierer**) setzen die **Features** (die einzelnen Funktionen und Merkmale der App) in **Quellcode** (der von Menschen lesbare Text, in dem ein Programm geschrieben ist) um. Dies geschieht typischerweise in kurzen, zeitlich begrenzten Entwicklungsabschnitten, den sogenannten **Sprints** (ein Begriff aus dem agilen **Scrum**-Framework).
- **Test:** Das **Quality Assurance (QA) Team** (die Abteilung, die für die Sicherstellung der Produktqualität durch Tests zuständig ist) führt umfangreiche Tests durch. Hierzu gehören **automatisierte Tests** (vom Computer selbst ausgeführte Prüfroutinen) und **manuelle Tests** (von Testern durchgeführte Prüfungen).
- **Deployment:** Die finale, fehlerbereinigte Version der App wird in den **App Store** (z.B. Apple App Store oder Google Play Store) oder auf **Servern** (leistungsstarke Computer, die Dienste für andere Computer, die **Clients**, bereitstellen) **bereitgestellt**. Dieser Vorgang wird auch als **Rollout** (die schrittweise oder vollständige Einführung eines Produkts oder Updates) bezeichnet.
- **Wartung:** Nach dem offiziellen **Release** (der Veröffentlichung der Software) wird der Betrieb gesichert. Dazu gehören das Beheben von Fehlern (**Bugfixing**) und die Entwicklung neuer Funktionalitäten.

1.3.3 Erklärender Text

Die Hauptphasen Des SDLC

Der **Softwareentwicklungsprozess (SDLC – Software Development Life Cycle)** ist in folgende, logisch aufeinander aufbauende Hauptphasen unterteilt, die zusammen den vollständigen Lebenszyklus eines Softwaresystems darstellen:

Phase	Fokusfrage	Zentrale Tätigkeiten	Ergebnisse/Artefakte
1. Anforderungsanalyse	**WAS** soll die Software leisten?	Stakeholder-Interviews, Erstellung von User Stories, Abgrenzung des **Scopes**.	**Pflichtenheft** (aus Sicht des Entwicklers), **Lastenheft** (aus Sicht des Kunden).
2. Entwurf/Modellierung	**WIE** wird die Software aufgebaut?	efinition der Software-Architektur, Datenbank-Design, Erstellung von UML-Diagrammen, Definition von Schnittstellen (APIs).	**Architektur-Dokumentation**, Entwurfs-Modelle.
3. Implementierung	Das **CODING**	Schreiben des Quellcodes, Einhaltung von **Coding Conventions**, regelmäßige Code-Reviews.	**Quellcode**, Einzelne Programm-Module.
4. Test	Funktioniert die Software wie erwartet?	**Unit-Tests** (Prüfung einzelner Funktionen), **Integrationstests** (Prüfung des Zusammenspiels), **Systemtests**, **Akzeptanztests** (Prüfung durch den Kunden).	**Testergebnisse**, Liste behobener Fehler.

5. Deployment	Bereitstellung für den Endnutzer	Installation auf Zielsystemen (Servern, App Stores), Konfiguration der Umgebung, **Continuous Integration/Deployment (CI/CD)**.	**Veröffentlichte Softwareversion**.
6. Wartung	Sicherung des Betriebs und Weiterentwicklung	**Bugfixing** (Fehlerkorrekturen), **Refactoring** (Code-Verbesserung ohne Funktionsänderung), Implementierung neuer Anforderungen (geht zurück zu Phase 1).	**Updates, Patches**.

Ausführliche Erklärungen der Phasen und Begriffe

Anforderungsanalyse

In dieser initialen Phase geht es darum, die **Bedürfnisse des Kunden oder der Endbenutzer zu verstehen und zu dokumentieren**.

- **Stakeholder-Interviews:** Strukturierte Gespräche mit allen **Stakeholdern** (Interessengruppen), um deren Erwartungen und Anforderungen zu erfassen.
- **User Stories:** Kurze, aus Benutzersicht formulierte Beschreibungen von Funktionen (siehe Praxisbeispiel).
- **Abgrenzung des Scopes:** Die klare Festlegung der **Projektgrenzen** (was gehört zum Projekt und was nicht?), um das gefürchtete **Scope Creep** zu vermeiden.
- **Lastenheft:** Ein Dokument, das die **Gesamtheit der Anforderungen aus Sicht des Auftraggebers** (Was wird gewünscht?) beschreibt.
- **Pflichtenheft:** Ein Dokument, das die Anforderungen **aus Sicht des Auftragnehmers/Entwicklers** (Wie wird es umgesetzt?) detailliert beschreibt und konkretisiert.

Entwurf/Modellierung

Diese Phase legt das **Fundament der technischen Lösung**. Die Frage ist nicht *was* die App können soll, sondern *wie* sie technisch funktioniert.

- **Software-Architektur:** Die grundlegende Struktur des Softwaresystems, die die wichtigsten Komponenten, deren Beziehungen und die Prinzipien ihrer Gestaltung beschreibt (z. B. 3-Schicht-Architektur).
- **Datenbank-Design:** Der Entwurf der Struktur, Organisation und Speicherung der Daten, oft mithilfe von **ER-Diagrammen** (Entity-Relationship-Diagrammen).
- **APIs (Application Programming Interfaces):** Definierte Schnittstellen, die die Interaktion zwischen verschiedenen Softwarekomponenten oder Systemen regeln.

Implementierung

Dies ist die Phase der **eigentlichen Programmierung**.

- **Quellcode:** Der in einer Programmiersprache geschriebene Code, der später in Maschinencode übersetzt wird.
- **Coding Conventions:** Formale Regeln und Richtlinien zur Formatierung und Stilistik des Quellcodes, um die Lesbarkeit und Wartbarkeit im Team zu gewährleisten.
- **Code-Reviews:** Systematische Überprüfung des Quellcodes durch andere Entwickler, um Fehler zu identifizieren, die Qualität zu sichern und Wissen zu teilen.

Test

Diese Phase dient der **Überprüfung der Software auf Fehlerfreiheit und Konformität mit den Anforderungen**.

- **Unit-Tests:** Tests, die die **kleinsten testbaren Einheiten** des Codes (z. B. einzelne Funktionen oder Methoden) isoliert prüfen.
- **Integrationstests:** Tests, die das **Zusammenspiel** mehrerer Komponenten oder Module prüfen (z. B. die Verbindung zwischen Web-Anwendung und Datenbank).
- **Akzeptanztests:** Formelle Tests, die vom Kunden oder Endnutzer (oder in deren Auftrag) durchgeführt werden, um die **endgültige Freigabe** für die Veröffentlichung zu erteilen.

Deployment

Die Phase der **Bereitstellung** der Anwendung für die Endnutzer.

- **Continuous Integration/Deployment (CI/CD):** Eine agile Methodik, bei der Code-Änderungen häufig (CI) und automatisch in das Haupt-Repository integriert, getestet und bei erfolgreicher Prüfung automatisch auf dem Zielsystem bereitgestellt (CD) werden. Dies beschleunigt den gesamten Prozess.

6. Wartung

Die Phase, die nach dem Release beginnt und oft den **längsten Teil des SDLC** ausmacht.

- **Bugfixing:** Die Korrektur von Fehlern (**Bugs**) in der produktiven Software.

- **Refactoring:** Die interne Umstrukturierung und **Verbesserung der Codequalität** (z.B. Lesbarkeit, Performance) **ohne Änderung des externen Verhaltens oder der Funktionalität** der Software.

1.3.4 Vorgehensmodelle im Überblick

Diese Phasen können unterschiedlich durchlaufen werden:

Die Art und Weise, wie die oben beschriebenen Phasen durchlaufen werden, wird durch das gewählte **Vorgehensmodell** definiert:

- **Klassische Modelle (z. B. Wasserfallmodell):**
  - **Beschreibung:** Die Phasen werden streng **sequenziell** (nacheinander) durchlaufen. Eine Phase muss vollständig **abgeschlossen und abgenommen** sein, bevor die nächste beginnt.
  - **Charakteristik: Starr und wenig flexibel** für Änderungen. Änderungen an den Anforderungen sind nach Phasenbeginn sehr schwierig und teuer umzusetzen.
- **Agile Modelle (z. B. Scrum, Kanban):**
  - **Beschreibung:** Die Entwicklung erfolgt **iterativ** (wiederholend) und **inkrementell** (schrittweise). Der gesamte Prozess wird in kurze, sich wiederholende Zyklen (Iterationen) aufgeteilt.
  - **Charakteristik:** Hohe **Flexibilität** für sich ändernde Anforderungen, ständige **Rückkopplung mit dem Kunden** und frühe Lieferung von funktionsfähigen Teilprodukten. **Scrum** nutzt hierfür die festen **Sprints**. **Kanban** fokussiert sich auf einen kontinuierlichen Flow und die Limitierung laufender Arbeiten (**Work in Progress**).

1.3.5 Fragen zur Vertiefung

1. Welche Phasen gehören zum Softwareentwicklungsprozess, und in welcher Phase werden Fehler am günstigsten behoben?

   Die sechs Phasen des **SDLC** sind: **1. Anforderungsanalyse, 2. Entwurf/Modellierung, 3. Implementierung, 4. Test, 5. Deployment, 6. Wartung**. Fehler werden am günstigsten in den **frühen Phasen** (**Anforderungsanalyse** oder **Entwurf/Modellierung**) behoben.

2. Was ist der Unterschied zwischen dem **Lastenheft** (Phase 1) und der **Architektur-Dokumentation** (Phase 2)?

Das **Lastenheft** (Phase 1) definiert **WAS** die Software leisten soll, aus Sicht des Kunden. Die **Architektur-Dokumentation** (Phase 2) definiert **WIE** die Software technisch aufgebaut wird, aus Sicht des Entwicklerteams.

3. Wie unterscheiden sich klassische (z. B. Wasserfall) und agile (z. B. Scrum) Vorgehensmodelle in Bezug auf die **Flexibilität** und die **Kundenintegration**?

   Klassische Modelle sind **sequenziell** und **starr**, was die Flexibilität für Änderungen gering macht. Die Kundenintegration ist meist auf den Anfang und das Ende beschränkt. **Agile Modelle** sind **iterativ** und bieten **hohe Flexibilität**. Die Kundenintegration ist **ständig** (z. B. in kurzen Entwicklungszyklen, den **Sprints**).